How Big Is an LLM?

Calculating Parameter Counts in Modern Large Language Models

CS 115B – Spring 2026

James Pustejovsky

March 20, 2026

Roadmap

1. Why Parameter Count Matters
2. Base Transformer: Parameter Anatomy
3. Modern Architectural Variants
4. Fine-Tuning and RLHF: Parameter Overhead
5. Comparison Summary
6. Take-Aways

Why Parameter Count Matters

What parameters *are*

- Learnable weights stored in GPU/TPU memory
- Determine **model capacity** and expressivity
- Drive compute cost for both training and inference
- Set minimum memory footprint (FP32: 4 bytes/param; BF16: 2 bytes/param)

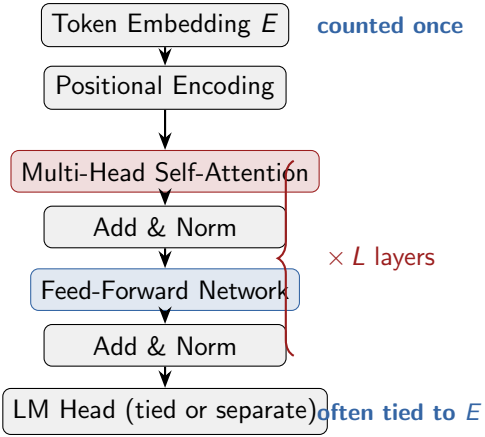
The scaling intuition

Doubling d quadruples attention parameters.
Doubling L doubles everything else.

Representative sizes (2024 – 2025)

Model	Params
GPT-2	1.5 B
LLaMA-3 8B	8 B
LLaMA-3 70B	70 B
Mistral 7B	7 B
Gemini 1.5 Pro	$\sim$340 B
GPT-4 (est.)	$\sim$1.8 T

Transformer Architecture at a Glance



Example Hyperparameters (800B-scale Model)

Architectural choices

Hyperparameter	Symbol	Value
Vocabulary size	V	50,000
Hidden dimension	d	16,384
FF expansion factor	$\times$	4
FF dimension	d_{ff}	65,536
Number of layers	L	250

- Layer normalisation parameters are negligible ($2d$ per layer)
- Biases omitted for clarity; they add $< 0.1\%$
- These values are chosen to produce a model in the $\sim$**800B** range

Component 1 – Token Embedding Table

Definition

Each of the V tokens is mapped to a d-dimensional vector.

$$\text{Embedding params} = V \times d = 50{,}000 \times 16{,}384 = \mathbf{819{,}200{,}000} \approx 0.82\,\text{B}$$

- Stored as a matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$
- The **LM head** (output projection to logits) is frequently *weight-tied* to $\mathbf{E}$, adding zero extra parameters
- If untied: another $d \times V \approx 0.82\,\text{B}$ is added
- Positional encodings (learned RoPE / ALiBi) add at most ctx_len $\times\, d$ – small relative to $V \times d$

Takeaway

The embedding table is large in absolute terms but *tiny* compared to the transformer blocks.

Component 2a – Multi-Head Self-Attention

Q, K, V projections

Three weight matrices, each $d \times d$, usually fused:

$$W_{QKV} : d \times 3d$$

$$= 16{,}384 \times 49{,}152 = 805{,}306{,}368$$

Output projection

$$W_O : d \times d = 16{,}384^2 = 268{,}435{,}456$$

Total per-layer attention params

$$\underbrace{805{,}306{,}368}_{W_{QKV}} + \underbrace{268{,}435{,}456}_{W_O}$$

$$= \mathbf{1{,}073{,}741{,}824} \approx 1.07\,\text{B}$$

Heads don't add params

Splitting d into h heads merely partitions the same matrices – no extra parameters.

Component 2b – Feed-Forward Network

Two-layer MLP with expansion

Layer 1: $d \times d_{\text{ff}} = 16{,}384 \times 65{,}536$

$= 1{,}073{,}741{,}824$

Layer 2: $d_{\text{ff}} \times d = 65{,}536 \times 16{,}384$

$= 1{,}073{,}741{,}824$

Total per-layer FFN params

$2 \times 1{,}073{,}741{,}824 = \mathbf{2{,}147{,}483{,}648} \approx 2.15\,\text{B}$

SwiGLU / GLU variants

Models using gated MLPs (LLaMA, Gemma) have *three* matrices: $W_1, W_2, W_3 \in \mathbb{R}^{d \times d_{\text{ff}}}$, giving $3 \times dd_{\text{ff}} \approx 3.22\,\text{B}$ per layer.

Component 2c – Parameters per Transformer Layer

Summing attention + FFN

$$\text{Per-layer params} = \underbrace{1{,}073{,}741{,}824}_{\text{Attention}} + \underbrace{2{,}147{,}483{,}648}_{\text{FFN}} = \mathbf{3{,}221{,}225{,}472} \approx 3.22\,\text{B}$$

Breakdown (pie)

Sub-layer	Params	%
Attention	1.07B	33%
FFN	2.15B	67%
LayerNorm	$\sim$0	–

Key ratio

FFN always dominates attention by roughly **2 : 1** under the standard $\times 4$ expansion. With SwiGLU ($\times 4$, 3 gates) it shifts to **3 : 1**.

Grand Total – 800B Scale

All transformer layers

$$L \times \text{per-layer} = 250 \times 3{,}221{,}225{,}472 = 805{,}306{,}368{,}000 \approx 805.3\,\text{B}$$

Adding the embedding table

$$\text{Total} \approx 805.3\,\text{B} + 0.82\,\text{B} = \textbf{806.1 B parameters}$$

Component	**Params**	**Share**
Embedding table	~ 0.82 B	0.1%
250× Attention blocks	~ 268 B	33%
250× FFN blocks	~ 537 B	67%
Total	$\sim$ **806** B	**100%**

Grouped Query Attention (GQA / MQA)

Standard MHA

h heads, each with its own Q, K, V:

$$W_{QKV} : d \times 3d \quad (= d \times 3d)$$

Multi-Query Attention (MQA)

All Q heads share *one* K and V head:

$$W_Q : d \times d, \quad W_{KV} : d \times 2d_h$$

Saving: KV matrices shrink by factor h.

Grouped Query Attention (GQA)

g groups; each group shares K, V:

Memory impact at inference

KV cache scales with the number of K/V heads, not Q heads. GQA reduces **KV cache memory** by $h/g\times$ with minimal quality loss (Ainslie et al., 2023).

Parameter comparison ($d = 4096$, $h = 32$, $g = 8$)

Variant	**Attn params**
MHA	~ 67 M
GQA ($g = 8$)	~ 42 M
MQA ($g = 1$)	~ 35 M

Mixture of Experts (MoE)

Architecture

Replace each dense FFN with E expert FFNs plus a router:

$$\text{Expert}_i : d \to d_{\text{ff}} \to d \quad (i = 1 \dots E)$$

$$\text{Router} : d \to E \quad \text{(softmax over experts)}$$

Only K experts are activated per token (*top-K routing*).

Parameter count

$$\text{MoE FFN} = E \times 2\, d\, d_{\text{ff}} + d \times E$$

Total params grow by $\sim E\times$, but **FLOPs** grow

Real examples

Model	Total	Active
Mixtral 8×7B	47B	13B
Mixtral 8×22B	141B	39B
GPT-4 (est.)	∼1.8T	∼220B
Grok-1	314B	86B

Key insight

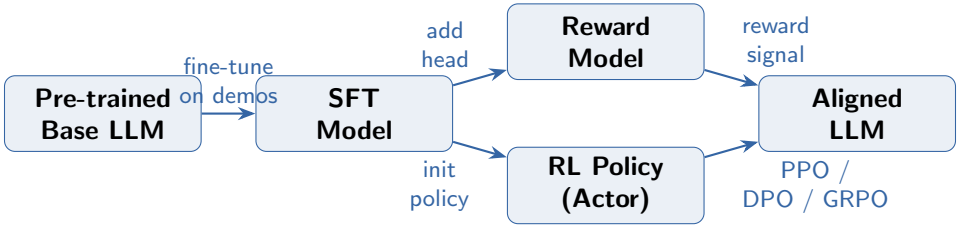
MoE decouples **model capacity** (total params) from **inference cost** (active params per token).

Quick Comparison – Modern 7B / 70B Configs

Model	d	L	h (Q/KV)	**FFN type**	**Params**
LLaMA-3 8B	4096	32	32/8	SwiGLU	8.0B
LLaMA-3 70B	8192	80	64/8	SwiGLU	70.6B
Mistral 7B	4096	32	32/8	SwiGLU	7.2B
Gemma 7B	3072	28	16/16	GeGLU	8.5B
Falcon 40B	8192	60	64/8	Std MLP	40.0B
Our 800B example	16384	250	– / –	Std $\times 4$	806B

- SwiGLU / GeGLU models use three FFN matrices but a smaller expansion factor ($\approx \frac{8}{3}d$) to keep FLOP counts comparable to a standard $\times 4$ expansion
- GQA (8 KV heads vs. 64 Q heads in LLaMA-3 70B) cuts KV cache by $8\times$ at inference

Fine-Tuning Pipeline Overview



Each dashed box is a **separate copy** of the base model

Stage 1 – Supervised Fine-Tuning (SFT)

What changes?

- **Same architecture** as the pre-trained base model
- All (or a subset of) weights updated on curated demonstration data
- **Parameter count: identical to base**

Parameter-efficient SFT (LoRA)

Add low-rank adapters $\mathbf{A} \in \mathbb{R}^{d \times r}$, $\mathbf{B} \in \mathbb{R}^{r \times d}$ to each weight matrix:

$$\Delta W = \mathbf{BA}, \quad r \ll d$$

$$\text{LoRA params per layer} = 2 \times d \times r$$

Memory during SFT

Full SFT: **3 optimizer states** (params + gradient + Adam m, v)
$\Rightarrow \sim$ 16 bytes/param in FP32.

A 70B model needs $\sim$ 1.1 TB of optimizer memory – typically handled via ZeRO / FSDP sharding.

LoRA overhead (rank 16)

Model	Trainable %
LLaMA-3 8B	$\sim 0.1\%$
LLaMA-3 70B	$\sim 0.07\%$

Stage 2 – Reward Modeling (for PPO)

Architecture

Start from the SFT model; replace the LM head with a **scalar reward head**:

$$\underbrace{\text{SFT backbone}}_{\theta_{\text{base}}} + \underbrace{W_r \in \mathbb{R}^{d\times 1}}_{\text{reward head}}$$

Parameter count

$$\text{RM params} = \text{base params} + d$$

The reward head is negligible ($d = 16{,}384$ vs. 806 B).

In practice: RM is a full copy of the base model with one extra linear layer.

Training objective

Trained on preference pairs (y^+, y^-):

$$\mathcal{L}_{\text{RM}} = -\log \sigma\big(r(x, y^+) - r(x, y^-)\big)$$

The RM must be **at least as large** as the policy to give useful signal (Ziegler et al., 2019; Bai et al., 2022).

Stage 3a – PPO: Proximal Policy Optimization

Four models in memory simultaneously

Model	Init from	Updated?	Params
Actor (policy)	SFT model	Yes (PPO)	θ
Critic (value fn.)	SFT or RM	Yes (PPO)	$\theta + d$
Reference policy	SFT model	No (frozen)	θ
Reward model	RM	No (frozen)	$\theta + d$
Total			$\approx 4 \times \theta$

Critic head

The value function (critic) appends one linear layer to the backbone:

Stage 3b – DPO: Direct Preference Optimization

Key idea (Rafailov et al., 2023)

Eliminate the reward model entirely. Optimize the policy *directly* on preference pairs (y^+, y^-):

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \beta \log\right.\right.$$

Models in memory

Model	Updated?	Params
Policy π_θ	Yes	θ
Ref. policy π_{ref}	No	θ
Total		$\approx 2\theta$

DPO advantage

Half the memory of PPO. No separate RM training run. No critic.

- The reference model is kept frozen to supply the KL penalty implicitly
- In memory-constrained settings, the reference model can be quantised (e.g., QLORA +

Stage 3c – GRPO: Group Relative Policy Optimization

Key idea (Shao et al., 2024 – DeepSeek-Math)

Remove the **critic / value model** from PPO. Estimate advantage from group-relative rewards:

$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

where $\mathbf{r} = \{r_1, \ldots, r_G\}$ are rewards for G sampled outputs for the same prompt x.

Models in memory

Model	Updated?	Params
Policy π_θ	Yes	θ
Ref. policy	No	θ
Reward model	No	θ
Total		$\approx 3\theta$

GRPO loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G} \min\Big(\rho_i \hat{A}_i,\ \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

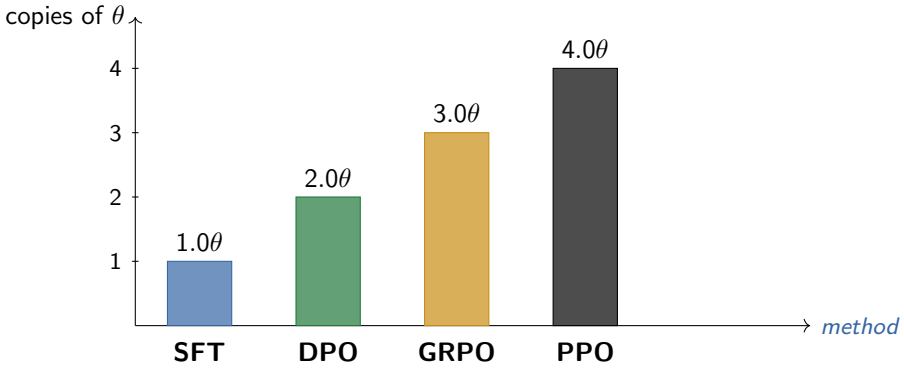
RLHF Method Comparison

Method	Reward Model	Critic Model	Ref. Policy	Copies	Notes
SFT only	–	–	–	1θ	No RL
PPO	✓	✓	✓	4θ	Most stable
DPO	×	×	✓	2θ	No RM needed
GRPO	✓	×	✓	3θ	No critic
REINFORCE	✓	×	×	2θ	High variance

Memory cost at 70B scale

Method	Total params	BF16 footprint (approx.)
SFT	70B	∼140 GB

Total Parameter Overhead – A Unified View



- **SFT**: one model, standard cross-entropy training
- **DPO**: policy + frozen reference – no RM, no critic
- **GRPO**: policy + reference + reward model – no critic
- **PPO**: all four models – most memory-intensive

Take-Aways

1. **FFN dominates**: two-thirds of each layer's parameters live in the feed-forward sublayer, not attention.
2. **Depth multiplies**: 250 layers $\times$ 3.22B $=$ 805B. Each additional layer is expensive in absolute terms.
3. **GQA saves KV cache**, not total params – crucial for long-context inference.
4. **MoE decouples capacity from FLOP cost**: you pay for all expert parameters in memory, but only K experts per forward pass.
5. **RL alignment multiplies model copies**: PPO is the most stable but the most memory-hungry ($4\times$); DPO and GRPO trade off stability for efficiency ($2\times$ – $3\times$).
6. **LoRA dramatically reduces trainable parameter count** for SFT (and can be applied to DPO / GRPO as well), but the frozen backbone still occupies memory.

References and Further Reading

- Vaswani, A. et al. (2017). *Attention is all you need*. NeurIPS 30.
- Ziegler, D. et al. (2019). *Fine-tuning language models from human preferences*. arXiv:1909.08593.
- Ainslie, J. et al. (2023). *GQA: Training generalised multi-query transformer models*. EMNLP 2023.
- Rafailov, R. et al. (2023). *Direct Preference Optimization: Your language model is secretly a reward model*. NeurIPS 2023.
- Shao, Z. et al. (2024). *DeepSeekMath: Pushing the limits of mathematical reasoning*. arXiv:2402.03300.
- Bai, Y. et al. (2022). *Training a helpful and harmless assistant with RLHF*. arXiv:2204.05862.